ODISHA ADARSHA VIDYALAYA SANGATHAN

TGT COMPUTER TEACHER RECRUITMENT EXAMINATION

MOST IMPORTANT QUESTIONS

Bitwise Operators

Concept of Bit Shifting | Left Shift (<<) | Right Shift (>>)
Uses & Limitations of Bitwise Operators | Bitwise Relational Operator Usage

204 Curated High-Yield, Exam-Oriented MCQs

Every Concept, Definition, Limitation, Assertion-Reason & Tricky Question Retained
Repetitive Numerical Drills Trimmed to the Most Representative Set

Strictly Aligned to the Prescribed Syllabus Boundary

Table of Contents

Section 1 — Introduction to Bitwise Operators	11 MCQs
Section 2 — Concept of Bit Shifting	15 MCQs
Section 3 — Left Shift Operator (<<)	27 MCQs
Section 4 — Right Shift Operator (>>)	27 MCQs
Section 5 — Uses of Bit Shift Operators	14 MCQs
Section 6 — Limitations of Bitwise Operators	15 MCQs
Section 7 — Bitwise Relational Operator Usage	16 MCQs
Section 8 — Binary and Numerical Questions	32 MCQs
Section 9 — Code and Output-Based Questions	15 MCQs
Section 10 — Difficult and Tricky Questions	15 MCQs
Section 11 — Mixed OAVS TGT Practice	17 MCQs

Total Questions in this Curated Set: 204

This set keeps every concept, definition, limitation, assertion-reason, match-the-following, and tricky question from the full bank, and a representative, non-repetitive spread of numerical/binary calculation and output-based questions across easy, moderate, and difficult levels — the combination most likely to appear in the actual OAVS TGT exam. Each question includes four options, the correct answer, and a short explanation, with correct answers balanced and randomised across A, B, C and D.

SECTION 1

Introduction to Bitwise Operators

Foundational concepts and terminology

Q1 | Concept | Bitwise Operators | Difficulty: Moderate

Bitwise operators in C operate directly on:

A. Whole numbers only

B. Strings of characters

C. Floating-point values only

D. Individual bits of operands

Ans: D

Explanation

Bitwise operators work at the level of individual binary digits (bits) of a value, not on the value as a whole.

Q2 | Concept | Bitwise Operators | Difficulty: Moderate

Bit shift operators change the value of an operand by:

A. Moving its bits left or right by a specified number of positions

B. Reversing the order of its digits

C. Rounding it to the nearest integer

D. Converting it into a character

Ans: A

Explanation

Bit shift operators reposition the bits of a value either to the left or to the right by a given number of positions.

Q3 | Concept | Bitwise Operators | Difficulty: Moderate

Which statement correctly describes bit shifting?

A. It converts an integer into a string

B. It always keeps the value exactly the same

C. It compares two values bit by bit without changing them

D. It moves each bit of a value to a new position, changing the value

Ans: D

Explanation

Bit shifting relocates bits within a value, which generally changes the numerical value.

Q4 | Concept | Bitwise Operators | Difficulty: Moderate

Fill in the blank: The two built-in bit shift operators in C are the left shift operator and the _____ operator.

A. right shift

B. modulus

C. increment

D. logical AND

Ans: A

Explanation

C provides two bit shift operators: left shift (<<) and right shift (>>).

Q5 | Definition | Bitwise Operators | Difficulty: Moderate

A bit shift operator can be best defined as an operator that:

A. Converts decimal numbers into hexadecimal

B. Adds two binary numbers together

C. Displaces the bits of a value by a fixed number of positions in a given direction

D. Deletes bits from a value permanently

Ans: C

Explanation

By definition, a bit shift operator moves (displaces) bits of its operand by a specified number of positions in one direction.

Q6 | Concept | Bitwise Operators | Difficulty: Moderate

Bit shifting is considered a low-level operation because it:

A. Cannot be used inside a C program

B. Only works on text data

C. Requires a separate compiler

D. Works directly on the binary representation of data

Ans: D

Explanation

Bit shifting manipulates the raw binary form of a value, which is why it is regarded as a low-level operation.

Q7 | Statement | Bitwise Operators | Difficulty: Moderate

Statement: Shift operators require exactly two operands — the value to be shifted and the number of positions to shift. Is this statement correct?

A. True only for left shift

B. False

C. True only for right shift

D. True

Ans: D

Explanation

Both << and >> need the value to shift and the count of positions, e.g., value << count.

Q8 | Concept | Bitwise Operators | Difficulty: Moderate

Which of the following correctly represents a left shift operation in C syntax?

A. 2 << x <<

B. x << >> 2

C. x >> 2

D. x << 2

Ans: D

Explanation

x << 2 shifts the bits of x left by 2 positions, which is the correct syntax.

Q9 | Concept | Bitwise Operators | Difficulty: Moderate

Why are bit shift operators considered efficient in programming?

A. Because shifting bits is a fast, low-level operation directly supported by hardware

B. Because they convert programs into binary automatically

C. Because they eliminate the need for any variables

D. Because they work without a compiler

Ans: A

Explanation

Shift operations correspond directly to fast hardware-level bit movement operations, making them efficient.

Q10 | Concept | Bitwise Operators | Difficulty: Moderate

In the expression a << b, which part specifies the number of positions to shift?

A. Both a and b equally

B. b

C. a

D. <<

Ans: B

Explanation

In a << b, 'a' is the value being shifted while 'b' specifies how many positions to shift it.

Q11 | Concept | Bitwise Operators | Difficulty: Moderate

Which of the following best distinguishes a bit shift operator from other operators covered in this chapter?

A. It always returns a Boolean result

B. It can only be used with characters

C. It moves bit positions instead of comparing or combining values

D. It cannot be used in any expression

Ans: C

Explanation

Unlike comparison, shift operators specifically reposition bits within the operand.

SECTION 2

Concept of Bit Shifting

Meaning, direction, and effect of shifting bits

Q12 | Concept | Bitwise Operators | Difficulty: Moderate

Bit shifting works by moving bits of a binary number in which direction(s)?

A. Either left or right, depending on the operator used

B. Only diagonally

C. Only right

D. Only left

Ans: A

Explanation

Depending on whether << or >> is used, bits are moved left or right respectively.

Q13 | Concept | Bitwise Operators | Difficulty: Moderate

When bits are shifted, positions that become empty are:

A. Automatically deleted

B. Left permanently blank

C. Filled with new bits (typically 0)

D. Replaced with the original value's last digit

Ans: C

Explanation

Vacated bit positions after a shift are filled in, usually with 0 in the simplest case.

Q14 | Concept | Bitwise Operators | Difficulty: Moderate

In the expression x << n, what does 'n' represent?

A. The number of variables in the expression

B. The final value of x

C. The number of bits in x

D. The number of positions by which the bits of x are shifted

Ans: D

Explanation

'n' specifies how many positions the bits of x should be moved.

Q15 | Numerical | Bitwise Operators | Difficulty: Moderate

Consider the number 3 (binary 0011 in 4 bits). After shifting its bits one position to the left, what binary pattern results (within 4 bits)?

A. 0001

B. 0110

C. 1001

D. 0011

Ans: B

Explanation

Shifting 0011 left by 1 position gives 0110, with a 0 filled in on the right.

Q16 | Numerical | Bitwise Operators | Difficulty: Moderate

Consider the number 12 (binary 1100 in 4 bits). After shifting its bits one position to the right, what binary pattern results?

A. 1100

B. 1001

C. 0011

D. 0110

Ans: D

Explanation

Shifting 1100 right by 1 position gives 0110, with a 0 filled in on the left.

Q17 | Concept | Bitwise Operators | Difficulty: Moderate

Shifting the bits of a value generally results in:

A. Exactly the same value as before

B. A new value that may be larger or smaller than the original

C. A value that is always zero

D. A value that is always negative

Ans: B

Explanation

Because shifting changes bit positions (and hence place values), the resulting number is typically different from the original.

Q18 | Numerical | Bitwise Operators | Difficulty: Moderate

If the number 40 (binary 00101000) is shifted right by 2 positions, what is the resulting binary value?

A. 10100000

B. 00101000

C. 00001010

D. 00010100

Ans: C

Explanation

00101000 shifted right by 2 becomes 00001010, which equals 10 in decimal.

Q19 | Concept | Bitwise Operators | Difficulty: Moderate

A shift operation is best understood as a way to:

A. Compare two values for equality

B. Add two separate variables together

C. Multiply or divide a value by powers of two through bit repositioning

D. Store a value permanently in memory

Ans: C

Explanation

Because binary place values are powers of two, shifting bits left or right effectively multiplies or divides by powers of two.

Q20 | Fill | Bitwise Operators | Difficulty: Moderate

Fill in the blank: Shifting bits to the left generally makes a value _____, while shifting bits to the right generally makes it _____.

A. smaller; larger

B. larger; smaller

C. negative; positive

D. unchanged; unchanged

Ans: B

Explanation

Left shifting moves bits toward higher place values (increasing the value), while right shifting moves them toward lower place values (decreasing the value).

Q21 | Concept | Bitwise Operators | Difficulty: Moderate

Conceptually, one left shift by a single position is equivalent to:

A. Halving the value

B. Doubling the value (multiplying by 2)

C. Squaring the value

D. Leaving the value unchanged

Ans: B

Explanation

Shifting bits one position left doubles the value, since each bit's place value becomes twice as large.

Q22 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the result of shifting the binary value 1 (0001) left by 3 positions?

A. 1000 (decimal 8)

B. 1100 (decimal 12)

C. 0001 (decimal 1)

D. 0100 (decimal 4)

Ans: A

Explanation

0001 shifted left by 3 positions becomes 1000, which is 8 in decimal — consistent with 1 x 2^3 = 8.

Q23 | Concept | Bitwise Operators | Difficulty: Moderate

Which of the following is a correct interpretation of a shifted value?

A. It represents the sum of the original value and the shift count

B. It represents the original value scaled by a power of two, in the shift direction

C. It represents the original value converted to a character

D. It represents an entirely unrelated random value

Ans: B

Explanation

A shift operation scales the original value by 2 raised to the power of the shift count, in the given direction.

Q24 | Numerical | Bitwise Operators | Difficulty: Moderate

The decimal value 6 is 0110 in 4-bit binary. What is the binary form after a left shift by 1 position (4-bit result, ignoring the bit shifted out)?

A. 1000

B. 1100

C. 0011

D. 0110

Ans: B

Explanation

0110 shifted left by 1 gives 1100 (the leading bit is shifted out of the 4-bit field).

Q25 | Concept | Bitwise Operators | Difficulty: Moderate

Which factor determines how much a value changes numerically due to a shift?

A. The number of positions shifted

B. The number of semicolons in the statement

C. The compiler being used

D. The variable's name

Ans: A

Explanation

The magnitude of change depends directly on how many positions the bits are shifted.

Q26 | Concept | Bitwise Operators | Difficulty: Moderate

Which of the following values, when shifted, illustrates that shifting can discard information?

A. A left shift where all shifted-out bits were 0

B. Any shift performed on the value 0

C. A shift by 0 positions

D. A right shift where the last bit shifted out was a 1

Ans: D

Explanation

When a right shift pushes out a bit that was 1, that bit of information is lost from the result.

SECTION 3

Left Shift Operator (<<)

Definition, working, syntax, numericals & applications

Q27 | Definition | Bitwise Operators | Difficulty: Moderate

The left shift operator in C is represented by the symbol:

A. <<

B. <

C. <=

D. >>

Ans: A

Explanation

The left shift operator is written as two less-than signs: <<.

Q28 | Definition | Bitwise Operators | Difficulty: Moderate

The left shift operator moves the bits of its left operand:

A. In a circular pattern with no bits lost

B. Toward the right, by the number of positions given as the right operand

C. Toward the left by exactly one position always

D. Toward the left, by the number of positions given as the right operand

Ans: D

Explanation

The << operator shifts bits of the left operand toward the left by the number of positions specified on its right.

Q29 | Syntax | Bitwise Operators | Difficulty: Moderate

Which of the following is valid C syntax for a left shift operation?

A. result = value << << positions;

B. result = << value positions;

C. result = value << positions;

D. result = positions << value <<;

Ans: C

Explanation

The correct syntax places the operand, then <<, then the shift count: value << positions.

Q30 | Working | Bitwise Operators | Difficulty: Moderate

During a left shift, what happens to bits shifted out from the left end?

A. They are discarded and lost

B. They are stored in a separate variable automatically

C. They wrap around to the right end

D. They cause a compilation error

Ans: A

Explanation

In a plain left shift, bits pushed out from the leftmost position are simply discarded.

Q31 | Working | Bitwise Operators | Difficulty: Moderate

During a left shift, what fills the vacated bit positions on the right?

A. 1 (one bits)

B. The original leftmost bit

C. Random bits

D. 0 (zero bits)

Ans: D

Explanation

Left shift always fills newly vacated positions on the right with 0.

Q32 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 3 << 1 in C?

A. 3

B. 12

C. 9

D. 6

Ans: D

Explanation

3 is 0011 in binary; shifted left by 1 gives 0110, which is 6.

Q33 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 2 << 3 in C?

A. 24

B. 16

C. 6

D. 8

Ans: B

Explanation

2 shifted left by 3 positions equals 2 x 2^3 = 16.

Q34 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 10 << 1 in C?

A. 5

B. 22

C. 20

D. 10

Ans: C

Explanation

10 is 1010 in binary; shifted left by 1 gives 10100, which is 20.

Q35 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 15 << 1 in C?

A. 45

B. 7

C. 15

D. 30

Ans: D

Explanation

15 is 1111 in binary; shifted left by 1 gives 11110, which is 30.

Q36 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 1 << 0 in C (shifting by zero positions)?

A. 2

B. Undefined always

C. 1

D. 0

Ans: C

Explanation

Shifting by 0 positions leaves the value unchanged, so 1 << 0 equals 1.

Q37 | Binary | Bitwise Operators | Difficulty: Moderate

What is the binary result of 2 << 2 (using 8-bit representation)?

A. 00001000

B. 00000010

C. 00010000

D. 00000100

Ans: A

Explanation

2 is 00000010; shifted left by 2 gives 00001000, which is 8 in decimal.

Q38 | Binary | Bitwise Operators | Difficulty: Moderate

The binary value 00000101 (decimal 5) is left shifted by 3 positions. What is the 8-bit result?

A. 00000101

B. 00010100

C. 00101000

D. 01010000

Ans: C

Explanation

00000101 shifted left by 3 becomes 00101000, equal to 40 in decimal.

Q39 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int x = 4;
printf("%d", x << 1);
```

A. 16

B. 4

C. 2

D. 8

Ans: D

Explanation

x << 1 shifts 4 (100) left by one position to get 1000, which is 8.

Q40 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 3, b;
b = a << 2;
printf("%d", b);
```

A. 6

B. 3

C. 12

D. 9

Ans: C

Explanation

a << 2 shifts 3 left by 2 positions: 3 x 2^2 = 12.

Q41 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int x = 1;
x = x << 3;
printf("%d", x);
```

A. 3

B. 1

C. 8

D. 6

Ans: C

Explanation

x << 3 shifts 1 left by 3 positions: 1 x 2^3 = 8.

Q42 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int n = 6;
printf("%d", n << 0);
```

A. 12

B. 3

C. 6

D. 0

Ans: C

Explanation

Shifting by 0 positions leaves the value unchanged, so the output is 6.

Q43 | Application | Bitwise Operators | Difficulty: Moderate

Left shifting is commonly used in programming to:

A. Quickly compare two strings

B. Quickly sort an array of integers

C. Quickly convert an integer to a floating-point number

D. Quickly multiply an integer by a power of two

Ans: D

Explanation

Because left shift by n multiplies a value by 2^n, it offers a fast way to multiply by powers of two.

Q44 | Application | Bitwise Operators | Difficulty: Moderate

In embedded and low-level programming, the left shift operator is often used to:

A. Convert a value into a string

B. Terminate a running loop

C. Set a bit at a particular position in a value

D. Delete a variable from memory

Ans: C

Explanation

Left shifting a value like 1 by n positions is a common technique to create a bit pattern with only the nth bit set.

Q45 | Limitation | Bitwise Operators | Difficulty: Moderate

A key limitation of the left shift operator is that it:

A. Cannot be used inside expressions

B. Can only be used with floating-point numbers

C. Always produces a negative result

D. Can cause the value to overflow beyond the range representable by its data type

Ans: D

Explanation

If shifting moves significant bits beyond the size of the data type, the resulting value can overflow and lose meaning.

Q46 | Limitation | Bitwise Operators | Difficulty: Moderate

Left shifting a value by too many positions may result in:

A. An automatic compilation error in every case

B. No change to the value at all

C. Loss of the original higher-order bits and an unexpected result

D. The value being converted to a character

Ans: C

Explanation

When the shift count is large enough, meaningful bits are pushed out of the value's storage, producing unexpected results.

Q47 | Error | Bitwise Operators | Difficulty: Moderate

What is wrong with the statement: int x = 5 << << 2;

A. The << operator is written twice, which is invalid syntax

B. Left shift cannot be assigned to a variable

C. The semicolon is misplaced

D. The value 5 cannot be shifted

Ans: A

Explanation

A shift expression should only contain one shift operator between the operand and the shift count; writing << twice is a syntax error.

Q48 | Assertion | Bitwise Operators | Difficulty: Moderate

```
Assertion (A): 4 << 1 gives the same result as 4 x 2.
Reason (R): A single left shift doubles the value of a non-negative integer.
```

A. Both A and R are true, but R does not explain A

B. A is false, but R is true

C. A is true, but R is false

D. Both A and R are true, and R correctly explains A

Ans: D

Explanation

4 << 1 equals 8, same as 4 x 2, because a single left shift doubles the value — R correctly explains A.

Q49 | Match | Bitwise Operators | Difficulty: Moderate

Match the following left shift expressions with their results:

```
(i) 2 << 1   (ii) 3 << 2   (iii) 1 << 5
(a) 32  (b) 4  (c) 12
```

A. (i)-a, (ii)-b, (iii)-c

B. (i)-b, (ii)-c, (iii)-a

C. (i)-b, (ii)-a, (iii)-c

D. (i)-c, (ii)-a, (iii)-b

Ans: B

Explanation

2<<1=4(b), 3<<2=12(c), 1<<5=32(a), giving the match (i)-b, (ii)-c, (iii)-a.

Q50 | Concept | Bitwise Operators | Difficulty: Moderate

Which of these expressions correctly doubles the value of variable x using a left shift?

A. x >> 1

B. x << 2

C. x << 0

D. x << 1

Ans: D

Explanation

Shifting left by exactly 1 position doubles the value of x.

Q51 | Concept | Bitwise Operators | Difficulty: Moderate

If x is a small positive integer, x << 3 is equivalent to:

A. x multiplied by 6

B. x multiplied by 8

C. x divided by 8

D. x multiplied by 3

Ans: B

Explanation

x << 3 multiplies x by 2^3 = 8.

Q52 | Scenario | Bitwise Operators | Difficulty: Moderate

A programmer wants to quickly compute 20 x 4 using a shift operation instead of multiplication. Which expression should be used?

A. 20 << 1

B. 20 >> 2

C. 20 << 2

D. 20 << 4

Ans: C

Explanation

Since 4 = 2^2, shifting 20 left by 2 positions (20 << 2) gives the same result as 20 x 4, which is 80.

Q53 | Concept | Bitwise Operators | Difficulty: Moderate

In the expression y << k, which operand is modified (has its bits moved)?

A. Neither; only the result changes

B. k

C. Both y and k equally

D. y

Ans: D

Explanation

y is the value being shifted; k merely specifies how many positions to shift it.

SECTION 4

Right Shift Operator (>>)

Definition, working, syntax, numericals & applications

Q54 | Definition | Bitwise Operators | Difficulty: Moderate

The right shift operator in C is represented by the symbol:

A. >

B. >=

C. <<

D. >>

Ans: D

Explanation

The right shift operator is written as two greater-than signs: >>.

Q55 | Definition | Bitwise Operators | Difficulty: Moderate

The right shift operator moves the bits of its left operand:

A. Toward the right by exactly one position always

B. In a circular pattern with no bits lost

C. Toward the left, by the number of positions given as the right operand

D. Toward the right, by the number of positions given as the right operand

Ans: D

Explanation

The >> operator shifts bits of the left operand toward the right by the number of positions specified on its right.

Q56 | Syntax | Bitwise Operators | Difficulty: Moderate

Which of the following is valid C syntax for a right shift operation?

A. result = value >> >> positions;

B. result = value >> positions;

C. result = >> value positions;

D. result = positions >> value >>;

Ans: B

Explanation

The correct syntax places the operand, then >>, then the shift count: value >> positions.

Q57 | Working | Bitwise Operators | Difficulty: Moderate

During a right shift, what happens to bits shifted out from the right end?

A. They are discarded and lost

B. They wrap around to the left end

C. They are automatically saved in another variable

D. They cause a runtime error

Ans: A

Explanation

In a plain right shift, bits pushed out from the rightmost position are simply discarded.

Q58 | Working | Bitwise Operators | Difficulty: Moderate

For a non-negative (unsigned) integer, what typically fills the vacated bit positions on the left during a right shift?

A. The original rightmost bit

B. Random bits

C. 1 (one bits)

D. 0 (zero bits)

Ans: D

Explanation

For non-negative integers, right shift fills the newly vacated positions on the left with 0.

Q59 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 8 >> 1 in C?

A. 4

B. 16

C. 2

D. 8

Ans: A

Explanation

8 is 1000 in binary; shifted right by 1 gives 0100, which is 4.

Q60 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 16 >> 3 in C?

A. 2

B. 1

C. 8

D. 4

Ans: A

Explanation

16 shifted right by 3 positions equals 16 / 2^3 = 2.

Q61 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 50 >> 1 in C?

A. 25

B. 26

C. 100

D. 24

Ans: A

Explanation

50 shifted right by 1 position equals 50 / 2 = 25.

Q62 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 7 >> 2 in C?

A. 0

B. 2

C. 1

D. 3

Ans: C

Explanation

7 is 0111 in binary; shifted right by 2 gives 0001, which is 1.

Q63 | Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 30 >> 0 in C (shifting by zero positions)?

A. 60

B. 30

C. 15

D. 0

Ans: B

Explanation

Shifting by 0 positions leaves the value unchanged, so 30 >> 0 equals 30.

Q64 | Binary | Bitwise Operators | Difficulty: Moderate

What is the binary result of 32 >> 3 (using 8-bit representation)?

A. 00000100

B. 00100000

C. 00001000

D. 00000010

Ans: A

Explanation

32 is 00100000; shifted right by 3 gives 00000100, which is 4 in decimal.

Q65 | Binary | Bitwise Operators | Difficulty: Moderate

The binary value 10000000 (decimal 128) is right shifted by 2 positions. What is the 8-bit result?

A. 01000000

B. 00010000

C. 10000000

D. 00100000

Ans: D

Explanation

10000000 shifted right by 2 becomes 00100000, equal to 32 in decimal.

Q66 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int x = 32;
printf("%d", x >> 2);
```

A. 8

B. 64

C. 4

D. 16

Ans: A

Explanation

x >> 2 shifts 32 right by two positions to get 8, since 32 / 2^2 = 8.

Q67 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 18, b;
b = a >> 1;
printf("%d", b);
```

A. 9

B. 36

C. 18

D. 8

Ans: A

Explanation

a >> 1 shifts 18 right by 1 position: 18 / 2 = 9.

Q68 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int x = 64;
x = x >> 3;
printf("%d", x);
```

A. 4

B. 8

C. 32

D. 16

Ans: B

Explanation

x >> 3 shifts 64 right by 3 positions: 64 / 2^3 = 8.

Q69 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int n = 15;
printf("%d", n >> 4);
```

A. 15

B. 0

C. -1

D. 1

Ans: B

Explanation

15 (binary 1111) shifted right by 4 positions moves all bits out, leaving 0.

Q70 | Application | Bitwise Operators | Difficulty: Moderate

Right shifting is commonly used in programming to:

A. Quickly convert an integer to a string

B. Quickly perform integer division by a power of two

C. Quickly compute the square root of a number

D. Quickly reverse the digits of a number

Ans: B

Explanation

Since right shift by n divides a value by 2^n, it provides a fast way to perform integer division by powers of two.

Q71 | Application | Bitwise Operators | Difficulty: Moderate

In low-level programming, the right shift operator is often used to:

A. Permanently delete a variable

B. Convert an integer into a floating-point number

C. Terminate a for loop

D. Extract or examine specific higher-order bits of a value

Ans: D

Explanation

Right shifting brings higher-order bits down to lower positions, making them easier to isolate and examine.

Q72 | Limitation | Bitwise Operators | Difficulty: Moderate

A key limitation of the right shift operator is that it:

A. Always increases the value

B. Can only be used with negative numbers

C. Cannot be used with variables, only constants

D. Discards the lower-order bits permanently, losing information

Ans: D

Explanation

Bits shifted out from the right end are lost, meaning any information stored there cannot be recovered.

Q73 | Limitation | Bitwise Operators | Difficulty: Moderate

Right shifting can produce results that surprise beginners mainly because:

A. It always produces a negative number

B. It changes the data type of the variable

C. The exact fractional part of the division is always discarded, not rounded

D. It never changes the value at all

Ans: C

Explanation

Right shift on integers performs a truncating (floor-like for positives) division, dropping the remainder rather than rounding.

Q74 | Error | Bitwise Operators | Difficulty: Moderate

What is wrong with the statement: int y = 10 >> >> 2;

A. 10 cannot be shifted at all

B. The semicolon is unnecessary

C. The >> operator is written twice, which is invalid syntax

D. Right shift requires two variables, not constants

Ans: C

Explanation

A shift expression should contain only one shift operator; writing >> twice results in a syntax error.

Q75 | Assertion | Bitwise Operators | Difficulty: Moderate

```
Assertion (A): 8 >> 1 gives the same result as 8 / 2.
Reason (R): A single right shift halves the value of a non-negative integer, discardin
g any remainder.
```

A. A is true, but R is false

B. Both A and R are true, and R correctly explains A

C. A is false, but R is true

D. Both A and R are true, but R does not explain A

Ans: B

Explanation

8 >> 1 equals 4, the same as 8 / 2, because a single right shift performs integer halving — R correctly explains A.

Q76 | Match | Bitwise Operators | Difficulty: Moderate

Match the following right shift expressions with their results:

```
(i) 20 >> 1   (ii) 32 >> 2   (iii) 9 >> 1
(a) 4  (b) 8  (c) 10
```

A. (i)-c, (ii)-b, (iii)-a

B. (i)-b, (ii)-c, (iii)-a

C. (i)-c, (ii)-a, (iii)-b

D. (i)-a, (ii)-b, (iii)-c

Ans: A

Explanation

20 >> 1 = 10 (c), 32 >> 2 = 8 (b), 9 >> 1 = 4 (a), giving the match (i)-c, (ii)-b, (iii)-a.

Q77 | Concept | Bitwise Operators | Difficulty: Moderate

Which of these expressions correctly halves the value of variable x using a right shift?

A. x >> 2

B. x >> 1

C. x >> 0

D. x << 1

Ans: B

Explanation

Shifting right by exactly 1 position halves the value of x (integer division).

Q78 | Concept | Bitwise Operators | Difficulty: Moderate

If x is a positive integer, x >> 4 is equivalent to:

A. x multiplied by 16

B. x divided by 8

C. x divided by 16 (integer division)

D. x divided by 4

Ans: C

Explanation

x >> 4 divides x by 2^4 = 16.

Q79 | Scenario | Bitwise Operators | Difficulty: Moderate

A programmer wants to quickly compute 96 / 8 using a shift operation instead of division. Which expression should be used?

A. 96 << 3

B. 96 >> 3

C. 96 >> 8

D. 96 >> 1

Ans: B

Explanation

Since 8 = 2^3, shifting 96 right by 3 positions (96 >> 3) gives the same result as 96 / 8, which is 12.

Q80 | Concept | Bitwise Operators | Difficulty: Moderate

In the expression m >> k, which operand's bits are relocated?

A. m

B. k

C. Neither; only the operator changes

D. Both m and k equally

Ans: A

Explanation

m is the value being shifted; k only specifies the number of positions.

SECTION 5

Uses of Bit Shift Operators

Practical applications of << and >>

Q81 | Use | Bitwise Operators | Difficulty: Moderate

One major practical use of the left shift operator is to:

A. Convert a value into a negative number

B. Multiply a value by a power of two quickly

C. Compare two variables for equality

D. Read input from the keyboard

Ans: B

Explanation

Left shift by n is functionally equivalent to multiplying by 2^n, making it a fast multiplication technique.

Q82 | Use | Bitwise Operators | Difficulty: Moderate

One major practical use of the right shift operator is to:

A. Store a value permanently on disk

B. Divide a value by a power of two quickly

C. Convert an integer to uppercase text

D. Concatenate two strings

Ans: B

Explanation

Right shift by n is functionally equivalent to integer division by 2^n.

Q83 | Use | Bitwise Operators | Difficulty: Moderate

Which of the following is a simple practical application of bit shifting?

A. Opening a file for reading

B. Quickly doubling a counter variable using << 1

C. Printing a string in reverse

D. Declaring a new function

Ans: B

Explanation

x << 1 is a simple, common way to double a variable's value quickly.

Q84 | Use | Bitwise Operators | Difficulty: Moderate

A shift operator can be used to quickly move a value's bits for the purpose of:

A. Repositioning bits to change the value in a predictable, calculable way

B. Making the value non-numeric

C. Erasing the value from memory entirely

D. Randomly changing a value with no predictable pattern

Ans: A

Explanation

Shifting relocates bits in a precise, predictable manner, allowing calculated changes to a value.

Q85 | Use | Bitwise Operators | Difficulty: Moderate

Which use case below correctly applies the right shift operator?

A. Computing 50 - 2 using 50 >> 1

B. Computing 50 + 2 using 50 >> 1

C. Computing 50 x 2 using 50 >> 1

D. Computing 50 / 2 using 50 >> 1

Ans: D

Explanation

50 >> 1 correctly computes 50 / 2 = 25 using a right shift.

Q86 | Scenario | Bitwise Operators | Difficulty: Moderate

A student wants to compute 7 x 8 using only a shift operator. Which expression achieves this?

A. 7 >> 8

B. 7 >> 3

C. 7 << 3

D. 7 << 8

Ans: C

Explanation

Since 8 = 2^3, 7 << 3 gives 7 x 8 = 56.

Q87 | Scenario | Bitwise Operators | Difficulty: Moderate

A student wants to compute 64 / 4 using only a shift operator. Which expression achieves this?

A. 64 << 4

B. 64 >> 2

C. 64 << 2

D. 64 >> 4

Ans: B

Explanation

Since 4 = 2^2, 64 >> 2 gives 64 / 4 = 16.

Q88 | Use | Bitwise Operators | Difficulty: Moderate

Bit shifting supports value modification at the binary level mainly because:

A. It changes the position of each bit, altering the place value it represents

B. It rewrites the entire program logic

C. It converts numbers into words

D. It changes the variable's data type automatically

Ans: A

Explanation

Every bit's contribution to a value depends on its position; shifting alters these positions directly.

Q89 | Use | Bitwise Operators | Difficulty: Moderate

A simple, direct application of bit shifting in binary-level manipulation is:

A. Reading a file line by line

B. Formatting text output

C. Generating powers of two, such as using 1 << n to get 2^n

D. Declaring a structure

Ans: C

Explanation

1 << n is a well-known technique to directly compute 2 raised to the power n.

Q90 | Numerical | Bitwise Operators | Difficulty: Moderate

Using the shifting technique, what value does 1 << 6 produce?

A. 6

B. 32

C. 64

D. 12

Ans: C

Explanation

1 << 6 computes 2^6, which equals 64.

Q91 | Numerical | Bitwise Operators | Difficulty: Moderate

Using the shifting technique, what value does 1 << 7 produce?

A. 64

B. 49

C. 14

D. 128

Ans: D

Explanation

1 << 7 computes 2^7, which equals 128.

Q92 | Use | Bitwise Operators | Difficulty: Moderate

Which of these is NOT a typical use of bit shift operators as covered in this syllabus?

A. Halving a value using a right shift

B. Sorting an array of strings alphabetically

C. Doubling a value using a left shift

D. Generating powers of two using a left shift

Ans: B

Explanation

String sorting is unrelated to bit shifting; the other three are direct, syllabus-relevant uses of shift operators.

Q93 | Scenario | Bitwise Operators | Difficulty: Moderate

To quickly generate the value 16 from 1 using a shift operator, which expression should be used?

A. 16 << 1

B. 1 << 16

C. 1 >> 4

D. 1 << 4

Ans: D

Explanation

1 << 4 equals 2^4 = 16.

Q94 | Use | Bitwise Operators | Difficulty: Moderate

Bit shifting can be used in simple binary-level manipulation to:

A. Convert an integer variable into a pointer

B. Change the compiler settings

C. Reposition a set bit from one place value to another

D. Permanently remove a bit from all future values

Ans: C

Explanation

Shifting simply moves a bit's position, which can relocate a set bit to a different place value.

SECTION 6

Limitations of Bitwise Operators

Restrictions, overflow, and common misconceptions

Q95 | Limitation | Bitwise Operators | Difficulty: Moderate

A general limitation of bitwise/shift operations is that they:

A. Always produce floating-point results

B. Can never be used inside loops

C. Work identically regardless of data type size

D. Depend on the size (in bits) of the data type being used

Ans: D

Explanation

The number of bits available in a data type limits how much information a shift operation can preserve.

Q96 | Limitation | Bitwise Operators | Difficulty: Moderate

A key restriction related to operands in shift operations is that:

A. The shift count should be a non-negative value less than the number of bits in the operand's type

B. The operand being shifted must always be negative

C. The shift count must always be exactly 1

D. Both operands must be floating-point values

Ans: A

Explanation

Shifting by a negative number or by a count greater than or equal to the bit-width of the type leads to undefined or unpredictable behavior.

Q97 | Limitation | Bitwise Operators | Difficulty: Moderate

Shifting can produce unexpected results when:

A. The operand being shifted is exactly zero

B. The shift operator is used inside a printf statement

C. The shift count is too large for the size of the data type

D. The shift count is exactly zero

Ans: C

Explanation

An excessively large shift count can move all significant bits out of range, giving results that do not match simple expectations.

Q98 | Limitation | Bitwise Operators | Difficulty: Moderate

One limitation related to data representation is that shifting:

A. Always increases the number of bits a value uses

B. Automatically converts integers into strings

C. Cannot be applied to any variable, only literals

D. Can silently discard bits, so the original information may not be recoverable

Ans: D

Explanation

Since shifted-out bits are dropped rather than stored, the original value may become impossible to fully reconstruct.

Q99 | Limitation | Bitwise Operators | Difficulty: Moderate

A common misconception corrected by understanding shift limitations is that:

A. Left shifting always gives a smaller result

B. Shift operators cannot be used with variables

C. Right shifting always gives a negative result

D. Left shifting always gives a valid, larger result, even though it may overflow

Ans: D

Explanation

Beginners often assume left shift always simply enlarges the value, without realizing that overflow can make the result invalid or unexpected.

Q100 | Limitation | Bitwise Operators | Difficulty: Moderate

Why is it considered a limitation that bit shift operators only work meaningfully on integer types?

A. It means they must always return zero

B. It restricts their direct use with floating-point data

C. It means they only work with character arrays

D. It means they cannot be used in any C program

Ans: B

Explanation

Since shifting depends on a fixed binary bit pattern, it is not meaningfully defined for floating-point representations in the way it is for integers.

Q101 | Limitation | Bitwise Operators | Difficulty: Moderate

Which of the following is a genuine limitation of the right shift operator specifically?

A. It cannot be assigned to a variable

B. It only works with even numbers

C. It can never be used more than once in a program

D. It permanently discards the lower-order bits that are shifted out

Ans: D

Explanation

Bits shifted out from the right end during a right shift are lost and cannot be retrieved.

Q102 | Limitation | Bitwise Operators | Difficulty: Moderate

Which of the following is a genuine limitation of the left shift operator specifically?

A. It can never be used with the value zero

B. It cannot shift by more than one position

C. It always produces a fractional result

D. It can push significant bits beyond the storage size, causing overflow

Ans: D

Explanation

If a left shift moves bits beyond the available storage width, the higher-order bits are lost, resulting in overflow.

Q103 | Statement | Bitwise Operators | Difficulty: Moderate

Statement: Shift operations are always completely safe to use with any shift count, without exception. Is this statement correct?

A. False

B. True, but only for left shift

C. True

D. True, but only for right shift

Ans: A

Explanation

Shift counts that are negative or too large for the data type's bit width can lead to undefined or unpredictable results, so this statement is false.

Q104 | Limitation | Bitwise Operators | Difficulty: Moderate

A restriction related to operands states that the value being shifted should be of a type that:

A. Is always a decimal fraction

B. Has a clearly defined, fixed number of bits

C. Has no defined size at all

D. Is always a string

Ans: B

Explanation

Shift operations rely on a well-defined bit-width to behave predictably, so the operand type should have a fixed number of bits.

Q105 | Limitation | Bitwise Operators | Difficulty: Moderate

Effects of excessive shifting can include:

A. The result becoming 0 (for left/right shift beyond bit-width) or otherwise unpredictable

B. The variable changing its name

C. The compiler adding extra memory automatically

D. The program automatically pausing execution

Ans: A

Explanation

Once the shift count meets or exceeds the number of bits in the type, results can become 0 or behave unpredictably depending on the compiler/platform.

Q106 | Limitation | Bitwise Operators | Difficulty: Moderate

Which situation best illustrates a limitation tied to data representation during shifting?

A. A variable being declared with a data type

B. A value's meaningful bits being shifted out and lost, changing its intended meaning

C. A value simply being printed to the screen

D. A loop counter being incremented

Ans: B

Explanation

When shifting removes meaningful bits, the resulting value may no longer represent what was originally intended.

Q107 | Misconception | Bitwise Operators | Difficulty: Moderate

A common misconception is that right shift by n and division by n give the same result. In reality:

A. Right shift by n is equivalent to division by 2 raised to the power n, not by n itself

B. Right shift by n is equivalent to division by n only when n is even

C. Right shift has no relationship with division at all

D. Right shift by n is always equivalent to division by n

Ans: A

Explanation

Right shift by n corresponds to dividing by 2^n, not by n — a frequent point of confusion for beginners.

Q108 | Misconception | Bitwise Operators | Difficulty: Moderate

A common mistake is assuming that shifting a value left always makes it strictly larger. Why can this be wrong?

A. Because if bits are shifted beyond the data type's size, the value can overflow and no longer be strictly larger in a meaningful sense

B. Because left shift always resets the value to 1

C. Because left shift never changes the value

D. Because left shift only works on negative numbers

Ans: A

Explanation

When overflow occurs, the bits that determine the 'larger' result are lost, so the assumption of always-increasing value does not hold reliably.

Q109 | Limitation | Bitwise Operators | Difficulty: Moderate

Keeping shift operations 'strictly within the scope of bitwise operators,' one general limitation is that they:

A. Replace the need for all other operators in a program

B. Cannot express operations like addition or comparison on their own; they only reposition bits

C. Work only inside comment lines

D. Automatically perform every arithmetic operation

Ans: B

Explanation

Shift operators are limited to moving bits; they do not themselves perform arithmetic like addition or perform comparisons.

SECTION 7

Bitwise Relational Operator Usage

Relational operators applied to shift expressions

Q110 | Concept | Bitwise Operators | Difficulty: Moderate

In the context of bitwise operations, a relational operator is typically used to:

A. Shift the bits of a value left or right

B. Compare the result of a shift expression with another value

C. Declare a new integer variable

D. Import a header file

Ans: B

Explanation

Relational operators such as ==, !=, <, and > are used to compare values, including the results produced by shift expressions.

Q111 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
if ((5 << 1) == 10)
    printf("Equal");
else
    printf("Not Equal");
```

A. Equal

B. Not Equal

C. 5

D. 10

Ans: A

Explanation

5 << 1 evaluates to 10, and 10 == 10 is true, so the program prints "Equal".

Q112 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
if ((8 >> 1) == 3)
    printf("Match");
else
    printf("No Match");
```

A. 3

B. No Match

C. 4

D. Match

Ans: B

Explanation

8 >> 1 evaluates to 4, and 4 == 3 is false, so the program prints "No Match".

Q113 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
if ((3 << 2) > 10)
    printf("Greater");
else
    printf("Not Greater");
```

A. 12

B. Greater

C. 10

D. Not Greater

Ans: B

Explanation

3 << 2 evaluates to 12, and 12 > 10 is true, so the program prints "Greater".

Q114 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
if ((20 >> 2) < 4)
    printf("Less");
else
    printf("Not Less");
```

A. Not Less

B. Less

C. 4

D. 5

Ans: A

Explanation

20 >> 2 evaluates to 5, and 5 < 4 is false, so the program prints "Not Less".

Q115 | Concept | Bitwise Operators | Difficulty: Moderate

Which relational operator would correctly check whether the result of x << 1 is not equal to 10?

A. >>

B. !=

C. ==

D. <<

Ans: B

Explanation

The != operator tests for inequality between the shift result and 10.

Q116 | Concept | Bitwise Operators | Difficulty: Moderate

In the expression (x >> 1) >= 5, which part is evaluated first according to C's precedence rules?

A. Both are evaluated simultaneously with no defined order

B. The shift expression x >> 1, due to the parentheses

C. The relational comparison >= 5, before the shift

D. Neither part is evaluated; it is invalid syntax

Ans: B

Explanation

Parentheses force x >> 1 to be evaluated first, and its result is then compared using >=.

Q117 | Scenario | Bitwise Operators | Difficulty: Moderate

A program needs to check if shifting the value 6 left by 1 position results in exactly 12. Which C condition correctly performs this check?

A. if ((6 >> 1) == 12)

B. if ((6 << 1) == 12)

C. if (6 << 1 = 12)

D. if ((6 << 1) << 12)

Ans: B

Explanation

6 << 1 equals 12, so comparing it with == 12 correctly checks the condition; parentheses ensure the shift happens before the comparison.

Q118 | Statement | Bitwise Operators | Difficulty: Moderate

Statement: Relational operators can be used together with shift expressions to test whether a shifted value meets a certain condition. Is this statement correct?

A. True

B. True, but only with right shift

C. True, but only with left shift

D. False

Ans: A

Explanation

Relational operators can compare the result of any valid expression, including both left shift and right shift expressions.

Q119 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
if ((10 >> 1) != 5)
    printf("Different");
else
    printf("Same");
```

A. 5

B. 10

C. Different

D. Same

Ans: D

Explanation

10 >> 1 evaluates to 5, and 5 != 5 is false, so the program prints "Same".

Q120 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
if ((2 << 3) <= 16)
    printf("Within Limit");
else
    printf("Exceeds Limit");
```

A. 16

B. Within Limit

C. Exceeds Limit

D. 8

Ans: B

Explanation

2 << 3 evaluates to 16, and 16 <= 16 is true, so the program prints "Within Limit".

Q121 | Concept | Bitwise Operators | Difficulty: Moderate

Why must a shift expression often be enclosed in parentheses when compared with a relational operator, such as in (x << 1) == 10?

A. Because C does not allow shift operators inside if statements

B. Because parentheses convert the expression into a string

C. Because relational operators cannot be used with integers

D. To ensure the shift is fully evaluated before the relational comparison is applied, avoiding precedence-related errors

Ans: D

Explanation

Explicit parentheses remove any ambiguity about operator precedence, ensuring the shift result is computed before comparison.

Q122 | Error | Bitwise Operators | Difficulty: Moderate

What is the likely issue with the condition: if (x << 1 = 10)

A. It shifts by a negative number

B. It uses the assignment operator (=) instead of the relational equality operator (==)

C. It uses too many shift operators

D. It is missing the variable declaration for x

Ans: B

Explanation

A single = performs assignment, not comparison; a relational check for equality requires ==, as in (x << 1) == 10.

Q123 | Assertion | Bitwise Operators | Difficulty: Moderate

Assertion (A): Relational operators can meaningfully evaluate the result produced by a bit shift expression.

```
Reason (R): A shift expression such as x << n always evaluates to a numeric value, whi
ch relational operators can then compare.
```

A. A is false, but R is true

B. A is true, but R is false

C. Both A and R are true, and R correctly explains A

D. Both A and R are true, but R does not explain A

Ans: C

Explanation

Since a shift expression produces a numeric result, relational operators can validly compare that result against another value — R explains A correctly.

Q124 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 4;
if ((a << 2) == 16)
    printf("Yes");
else
    printf("No");
```

A. No

B. Yes

C. 8

D. 16

Ans: B

Explanation

a << 2 evaluates to 4 x 4 = 16, and 16 == 16 is true, so the program prints "Yes".

Q125 | Scenario | Bitwise Operators | Difficulty: Moderate

A program must print "Big" only if right-shifting 50 by 1 position gives a value greater than 20. Which condition is correct?

A. if ((50 >> 1) > 20)

B. if ((50 >> 1) < 20)

C. if (50 >> 1 > 20 >> 1)

D. if ((50 << 1) > 20)

Ans: A

Explanation

50 >> 1 equals 25, and comparing 25 > 20 with the correctly parenthesized shift gives the intended check.

SECTION 8

Binary and Numerical Questions

Extensive calculation-based practice

Q126 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 6 << 1 in C?

A. 12

B. 10

C. 6

D. 14

Ans: A

Explanation

6 in binary is 00000110. Shifting left by 1 position(s) gives 00001100, which equals 12 in decimal (i.e., 6 x 2^1 = 12).

Q127 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 2 << 4 in C?

A. 34

B. 2

C. 32

D. 8

Ans: C

Explanation

2 in binary is 00000010. Shifting left by 4 position(s) gives 00100000, which equals 32 in decimal (i.e., 2 x 2^4 = 32).

Q128 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 6 << 2 in C?

A. 24

B. 22

C. 1

D. 26

Ans: A

Explanation

6 in binary is 00000110. Shifting left by 2 position(s) gives 00011000, which equals 24 in decimal (i.e., 6 x 2^2 = 24).

Q129 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 10 << 2 in C?

A. 42

B. 40

C. 20

D. 38

Ans: B

Explanation

10 in binary is 00001010. Shifting left by 2 position(s) gives 00101000, which equals 40 in decimal (i.e., 10 x 2^2 = 40).

Q130 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 1 << 6 in C?

A. 0

B. 1

C. 64

D. 6

Ans: C

Explanation

1 in binary is 00000001. Shifting left by 6 position(s) gives 01000000, which equals 64 in decimal (i.e., 1 x 2^6 = 64).

Q131 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 2 << 6 in C?

A. 2

B. 126

C. 12

D. 128

Ans: D

Explanation

2 in binary is 00000010. Shifting left by 6 position(s) gives 10000000, which equals 128 in decimal (i.e., 2 x 2^6 = 128).

Q132 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 3 << 2 in C?

A. 14

B. 12

C. 0

D. 6

Ans: B

Explanation

3 in binary is 00000011. Shifting left by 2 position(s) gives 00001100, which equals 12 in decimal (i.e., 3 x 2^2 = 12).

Q133 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 17 << 1 in C?

A. 8

B. 34

C. 51

D. 17

Ans: B

Explanation

17 in binary is 00010001. Shifting left by 1 position(s) gives 00100010, which equals 34 in decimal (i.e., 17 x 2^1 = 34).

Q134 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 24 >> 1 in C?

A. 11

B. 14

C. 12

D. 24

Ans: C

Explanation

24 in binary is 00011000. Shifting right by 1 position(s) gives 00001100, which equals 12 in decimal (i.e., 24 / 2^1, discarding the remainder, = 12).

Q135 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 80 >> 4 in C?

A. 1280

B. 7

C. 5

D. 10

Ans: C

Explanation

80 in binary is 01010000. Shifting right by 4 position(s) gives 00000101, which equals 5 in decimal (i.e., 80 / 2^4, discarding the remainder, = 5).

Q136 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 45 >> 1 in C?

A. 24

B. 22

C. 45

D. 21

Ans: B

Explanation

45 in binary is 00101101. Shifting right by 1 position(s) gives 00010110, which equals 22 in decimal (i.e., 45 / 2^1, discarding the remainder, = 22).

Q137 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 128 >> 5 in C?

A. 6

B. 4

C. 3

D. 4096

Ans: B

Explanation

128 in binary is 10000000. Shifting right by 5 position(s) gives 00000100, which equals 4 in decimal (i.e., 128 / 2^5, discarding the remainder, = 4).

Q138 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 150 >> 1 in C?

A. 300

B. 77

C. 150

D. 75

Ans: D

Explanation

150 in binary is 10010110. Shifting right by 1 position(s) gives 01001011, which equals 75 in decimal (i.e., 150 / 2^1, discarding the remainder, = 75).

Q139 | Binary/Numerical | Bitwise Operators | Difficulty: Easy

What is the value of 55 >> 1 in C?

A. 55

B. 29

C. 27

D. 110

Ans: C

Explanation

55 in binary is 00110111. Shifting right by 1 position(s) gives 00011011, which equals 27 in decimal (i.e., 55 / 2^1, discarding the remainder, = 27).

Q140 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 224 >> 5 in C?

A. 44

B. 14

C. 224

D. 7

Ans: D

Explanation

224 in binary is 11100000. Shifting right by 5 position(s) gives 00000111, which equals 7 in decimal (i.e., 224 / 2^5, discarding the remainder, = 7).

Q141 | Binary/Numerical | Bitwise Operators | Difficulty: Moderate

What is the value of 240 >> 4 in C?

A. 17

B. 14

C. 240

D. 15

Ans: D

Explanation

240 in binary is 11110000. Shifting right by 4 position(s) gives 00001111, which equals 15 in decimal (i.e., 240 / 2^4, discarding the remainder, = 15).

Q142 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 5 is written in binary as 00000101. What is the binary representation (8-bit) after shifting it left by 2 position(s)?

A. 00000101

B. 00010100

C. 00011000

D. 00000001

Ans: B

Explanation

00000101 shifted left by 2 gives 00010100, which is 20 in decimal.

Q143 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 3 is written in binary as 00000011. What is the binary representation (8-bit) after shifting it left by 5 position(s)?

A. 00000000

B. 01100000

C. 00000011

D. 01100100

Ans: B

Explanation

00000011 shifted left by 5 gives 01100000, which is 96 in decimal.

Q144 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 11 is written in binary as 00001011. What is the binary representation (8-bit) after shifting it left by 1 position(s)?

A. 00011010

B. 00000101

C. 00010110

D. 00001011

Ans: C

Explanation

00001011 shifted left by 1 gives 00010110, which is 22 in decimal.

Q145 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 2 is written in binary as 00000010. What is the binary representation (8-bit) after shifting it left by 2 position(s)?

A. 00000000

B. 00000010

C. 00001100

D. 00001000

Ans: D

Explanation

00000010 shifted left by 2 gives 00001000, which is 8 in decimal.

Q146 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 80 is written in binary as 01010000. What is the binary representation (8-bit) after shifting it right by 2 position(s)?

A. 01000000

B. 01010000

C. 00010100

D. 00011000

Ans: C

Explanation

01010000 shifted right by 2 gives 00010100, which is 20 in decimal.

Q147 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 96 is written in binary as 01100000. What is the binary representation (8-bit) after shifting it right by 1 position(s)?

A. 11000000

B. 01100000

C. 00110100

D. 00110000

Ans: D

Explanation

01100000 shifted right by 1 gives 00110000, which is 48 in decimal.

Q148 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 160 is written in binary as 10100000. What is the binary representation (8-bit) after shifting it right by 3 position(s)?

A. 00010100

B. 00011000

C. 00000000

D. 10100000

Ans: A

Explanation

10100000 shifted right by 3 gives 00010100, which is 20 in decimal.

Q149 | Binary | Bitwise Operators | Difficulty: Moderate

The decimal value 48 is written in binary as 00110000. What is the binary representation (8-bit) after shifting it right by 2 position(s)?

A. 00110000

B. 11000000

C. 00001100

D. 00010000

Ans: C

Explanation

00110000 shifted right by 2 gives 00001100, which is 12 in decimal.

Q150 | Numerical | Bitwise Operators | Difficulty: Moderate

By how many positions must 4 be shifted using the << operator to obtain 32?

A. 2

B. 3

C. 5

D. 4

Ans: B

Explanation

4 << 3 evaluates to 32, so 3 is the required number of shift positions.

Q151 | Numerical | Bitwise Operators | Difficulty: Moderate

By how many positions must 3 be shifted using the << operator to obtain 24?

A. 3

B. 5

C. 2

D. 4

Ans: A

Explanation

3 << 3 evaluates to 24, so 3 is the required number of shift positions.

Q152 | Numerical | Bitwise Operators | Difficulty: Moderate

By how many positions must 96 be shifted using the >> operator to obtain 12?

A. 3

B. 4

C. 2

D. 5

Ans: A

Explanation

96 >> 3 evaluates to 12, so 3 is the required number of shift positions.

Q153 | Numerical | Bitwise Operators | Difficulty: Moderate

By how many positions must 5 be shifted using the << operator to obtain 40?

A. 5

B. 4

C. 2

D. 3

Ans: D

Explanation

5 << 3 evaluates to 40, so 3 is the required number of shift positions.

Q154 | Numerical | Bitwise Operators | Difficulty: Easy

Which operator should be used with 6 to obtain the result 24?

A. ==

B. >>

C. <<

D. =

Ans: C

Explanation

Since 24 is greater than 6, the left shift operator (<<) is needed to reach 24 from 6.

Q155 | Numerical | Bitwise Operators | Difficulty: Easy

Which operator should be used with 8 to obtain the result 64?

A. <<

B. =

C. >>

D. ==

Ans: A

Explanation

Since 64 is greater than 8, the left shift operator (<<) is needed to reach 64 from 8.

Q156 | Numerical | Bitwise Operators | Difficulty: Easy

Which operator should be used with 3 to obtain the result 12?

A. <<

B. >>

C. =

D. ==

Ans: A

Explanation

Since 12 is greater than 3, the left shift operator (<<) is needed to reach 12 from 3.

Q157 | Numerical | Bitwise Operators | Difficulty: Easy

Which operator should be used with 9 to obtain the result 36?

A. <<

B. =

C. ==

D. >>

Ans: A

Explanation

Since 36 is greater than 9, the left shift operator (<<) is needed to reach 36 from 9.

SECTION 9

Code and Output-Based Questions

C code snippets and output prediction

Q158 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 5;
int b = a << 1;
printf("%d", b);
```

A. 5

B. 15

C. 10

D. 3

Ans: C

Explanation

Evaluating the shift expression in the code gives 10, which is printed by printf.

Q159 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 18;
int b = a >> 1;
printf("%d", b);
```

A. 8

B. 36

C. 9

D. 18

Ans: C

Explanation

Evaluating the shift expression in the code gives 9, which is printed by printf.

Q160 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int x = 40, y = 3;
printf("%d", x >> y);
```

A. 20

B. 5

C. 4

D. 10

Ans: B

Explanation

Evaluating the shift expression in the code gives 5, which is printed by printf.

Q161 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 64;
printf("%d", a >> 6);
```

A. 0

B. 32

C. 1

D. 2

Ans: C

Explanation

Evaluating the shift expression in the code gives 1, which is printed by printf.

Q162 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 7, b = 2;
int c = a << b;
printf("%d", c);
```

A. 28

B. 21

C. 14

D. 9

Ans: A

Explanation

Evaluating the shift expression in the code gives 28, which is printed by printf.

Q163 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int x = 10;
x = x << 1;
x = x << 1;
printf("%d", x);
```

A. 30

B. 20

C. 40

D. 80

Ans: C

Explanation

Evaluating the shift expression in the code gives 40, which is printed by printf.

Q164 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int n = 0;
printf("%d", n << 10);
```

A. 1024

B. 1

C. 10

D. 0

Ans: D

Explanation

Evaluating the shift expression in the code gives 0, which is printed by printf.

Q165 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 5;
int b = 2;
int c = (a << b) - a;
printf("%d", c);
```

A. 15

B. 25

C. 10

D. 20

Ans: A

Explanation

Evaluating the shift expression in the code gives 15, which is printed by printf.

Q166 | Output | Bitwise Operators | Difficulty: Moderate

What will the following code output?

```
int a = 12;
printf("%d %d", a >> 1, a >> 2);
```

A. 6 3

B. 3 6

C. 6 6

D. 12 6

Ans: A

Explanation

a >> 1 gives 12 / 2 = 6, and a >> 2 gives 12 / 4 = 3, so the output is "6 3".

Q167 | Output | Bitwise Operators | Difficulty: Difficult

What will the following code output?

```
int i;
for (i = 1; i <= 3; i++)
    printf("%d ", 1 << i);
```

A. 2 4 8

B. 1 2 3

C. 2 4 6

D. 1 4 8

Ans: A

Explanation

The loop prints 1<<1=2, 1<<2=4, and 1<<3=8 in sequence, giving "2 4 8".

Q168 | Output | Bitwise Operators | Difficulty: Difficult

What will the following code output?

```
int i;
for (i = 32; i >= 4; i = i >> 1)
    printf("%d ", i);
```

A. 32 16 8 4 2

B. 16 8 4

C. 32 8 4

D. 32 16 8 4

Ans: D

Explanation

Starting at 32, each iteration halves i via right shift: 32, 16, 8, 4; the loop stops once i falls below 4 after printing 4.

Q169 | Error | Bitwise Operators | Difficulty: Moderate

What is the compilation issue, if any, with the following code?

```
int a = 5
int b = a << 1;
printf("%d", b);
```

A. There is no issue; it compiles fine

B. The printf statement is invalid

C. The << operator is used incorrectly

D. Missing semicolon after 'int a = 5'

Ans: D

Explanation

The statement 'int a = 5' is missing a terminating semicolon, which causes a compilation error.

Q170 | Error | Bitwise Operators | Difficulty: Moderate

Identify the error in the following code:

```
int x = 10;
printf("%d", x << );
```

A. The semicolon is misplaced

B. printf cannot print integers

C. The variable x is undeclared

D. The shift count (right operand) is missing after <<

Ans: D

Explanation

The << operator requires a right operand specifying the shift count; leaving it blank is a syntax error.

Q171 | Output | Bitwise Operators | Difficulty: Difficult

What will the following code output?

```
int x = 4;
printf("%d", (x << 1) + (x >> 1));
```

A. 10

B. 6

C. 8

D. 12

Ans: A

Explanation

x << 1 gives 8 and x >> 1 gives 2; their sum, 8 + 2, equals 10.

Q172 | Scenario | Bitwise Operators | Difficulty: Moderate

A programmer writes int result = 9 << 1; but intended to compute 9 >> 1. What is the impact of this mistake?

A. The program crashes at runtime

B. The program computes 4 correctly despite the typo

C. The program computes 18 instead of the intended 4

D. The program will not compile

Ans: C

Explanation

9 << 1 evaluates to 18, while the intended 9 >> 1 would have evaluated to 4 — using the wrong operator changes the result significantly.

SECTION 10

Difficult and Tricky Questions

Advanced, multi-step, and conceptually tricky items

Q173 | Tricky | Bitwise Operators | Difficulty: Difficult

What will be the output of the following code?

```
int x = 1;
printf("%d", x << 1 << 1);
```

A. 8

B. 1

C. 4

D. 2

Ans: C

Explanation

Shift operators associate left to right: (x << 1) << 1 = (1 << 1) << 1 = 2 << 1 = 4.

Q174 | Tricky | Bitwise Operators | Difficulty: Difficult

What will be the output of the following code?

```
int x = 64;
printf("%d", x >> 1 >> 1);
```

A. 32

B. 16

C. 64

D. 8

Ans: B

Explanation

Shift operators associate left to right: (x >> 1) >> 1 = (64 >> 1) >> 1 = 32 >> 1 = 16.

Q175 | Tricky | Bitwise Operators | Difficulty: Difficult

Which of these expressions produces a result DIFFERENT from the other three?

A. 1 << 4

B. 6 << 1

C. 24 >> 1

D. 3 << 2

Ans: A

Explanation

6 << 1 = 12, 3 << 2 = 12, and 24 >> 1 = 12 all equal 12, whereas 1 << 4 = 16 is the odd one out.

Q176 | Tricky | Bitwise Operators | Difficulty: Moderate

A student claims that x << 1 and x * 2 always give identical results for any integer x in C. Under what condition can this claim fail?

A. When x is exactly equal to 1

B. When x is exactly equal to 0

C. When the left shift causes the value to overflow beyond the range of its data type

D. This claim never fails under any condition

Ans: C

Explanation

For most small positive values the two match, but if the shift overflows the data type's range, the shift result can diverge from ordinary multiplication.

Q177 | Tricky | Bitwise Operators | Difficulty: Moderate

What is the key difference in outcome between 5 << 2 and 5 >> 2?

A. 5 << 2 equals 1 while 5 >> 2 equals 20

B. Both expressions always equal 5

C. Both expressions always equal 10

D. 5 << 2 equals 20, while 5 >> 2 equals 1 — the two results move in opposite directions from 5

Ans: D

Explanation

5 << 2 = 20 (larger than 5), whereas 5 >> 2 = 1 (smaller than 5), showing the opposite effects of the two operators.

Q178 | Tricky | Bitwise Operators | Difficulty: Moderate

Which expression below evaluates to the SAME result as 24?

A. 6 << 1

B. 3 << 2

C. 3 << 3

D. 24 >> 1

Ans: C

Explanation

3 << 3 = 3 x 8 = 24, matching the target value; 3<<2=12, 24>>1=12, and 6<<1=12 do not.

Q179 | Tricky | Bitwise Operators | Difficulty: Moderate

In the expression (7 >> 1) << 1, what is the final result, and why might a beginner mistakenly expect it to equal 7?

A. The result is 7, because shifting right then left always cancels out

B. The result is 0, because the operations cancel to zero

C. The result is 6, not 7, because the right shift discards the last bit (a 1), which the following left shift cannot restore

D. The result is 14, because both shifts multiply the value

Ans: C

Explanation

7 >> 1 discards the least significant bit (1), giving 3; then 3 << 1 gives 6 — the lost bit means the original value 7 is not restored.

Q180 | Tricky | Bitwise Operators | Difficulty: Moderate

Evaluate: (2 << 3) >> 2. What is the result?

A. 4

B. 2

C. 8

D. 16

Ans: A

Explanation

2 << 3 = 16, and then 16 >> 2 = 4.

Q181 | Tricky | Bitwise Operators | Difficulty: Moderate

Evaluate: (40 >> 2) << 1. What is the result?

A. 40

B. 80

C. 10

D. 20

Ans: D

Explanation

40 >> 2 = 10, and then 10 << 1 = 20.

Q182 | Assertion | Bitwise Operators | Difficulty: Moderate

```
Assertion (A): (x << n) >> n does not always restore the original value of x.
Reason (R): Bits shifted out during the left shift due to overflow cannot be recovered
 by the subsequent right shift.
```

A. Both A and R are true, but R does not explain A

B. A is true, but R is false

C. A is false, but R is true

D. Both A and R are true, and R correctly explains A

Ans: D

Explanation

If the left shift causes bits to overflow out of the value's range, no later right shift can bring those lost bits back — R correctly explains A.

Q183 | Tricky | Bitwise Operators | Difficulty: Moderate

Which of the following pairs of expressions are numerically equal?

A. 3 << 1 and 3 >> 1

B. 16 >> 2 and 1 << 2

C. 8 >> 1 and 1 << 1

D. 16 >> 2 and 1 << 4

Ans: B

Explanation

16 >> 2 = 4 and 1 << 2 = 4, so this pair is equal; the other pairs do not match (16>>2=4 vs 1<<4=16; 8>>1=4 vs 1<<1=2; 3<<1=6 vs 3>>1=1).

Q184 | Tricky | Bitwise Operators | Difficulty: Moderate

A beginner writes 10 >> 1 expecting the answer 5.0 (a decimal). What is the actual output, and why?

A. 5, because integer right shift performs integer (whole-number) division, discarding any fractional part

B. 5.5, because the operator rounds to the nearest half

C. An error, because shifting cannot be applied to even numbers

D. 5.0, because C always keeps decimal precision in shifts

Ans: A

Explanation

Right shift on integers truncates any fractional remainder, so 10 >> 1 yields the whole number 5, not a decimal value.

Q185 | Tricky | Bitwise Operators | Difficulty: Moderate

Which single shift operation transforms the value 5 directly into 40?

A. 5 << 3

B. 5 << 4

C. 5 >> 3

D. 5 << 2

Ans: A

Explanation

5 << 3 = 5 x 8 = 40, matching the target.

Q186 | Tricky | Bitwise Operators | Difficulty: Moderate

Given int x = 3; consider two expressions: x << 1 and x << 1 << 1. What is the relationship between their results?

A. The results are unrelated

B. The second expression's result is exactly half the first

C. The second expression's result is exactly double the first

D. Both expressions give the same result

Ans: C

Explanation

x << 1 = 6, and x << 1 << 1 = (3 << 1) << 1 = 12, which is exactly double 6 — each further left shift doubles the value again.

Q187 | Tricky | Bitwise Operators | Difficulty: Moderate

Why can excessive right shifting of a small positive integer eventually always yield 0?

A. Because right shift is undefined for positive integers

B. Because right shift resets a value to 0 after exactly one use

C. Because each right shift discards the least significant bit, and enough repetitions remove all the value's set bits

D. Because right shift converts the value into a negative number

Ans: C

Explanation

Each right shift drops the lowest bit; after enough shifts, all originally set bits have been discarded, leaving 0.

SECTION 11

Mixed OAVS TGT Practice

Comprehensive mixed practice set

Q188 | Mixed | Bitwise Operators | Difficulty: Moderate

Which operator would you use to quickly multiply a number by 4?

A. >> 2

B. << 2

C. << 4

D. >> 4

Ans: B

Explanation

Multiplying by 4 corresponds to 2^2, so shifting left by 2 positions (<< 2) achieves this.

Q189 | Mixed | Bitwise Operators | Difficulty: Moderate

Which operator would you use to quickly divide a number by 8?

A. >> 3

B. << 8

C. >> 8

D. << 3

Ans: A

Explanation

Dividing by 8 corresponds to 2^3, so shifting right by 3 positions (>> 3) achieves this.

Q190 | Mixed | Bitwise Operators | Difficulty: Moderate

What is the result of 160 >> 5?

A. 5

B. 10

C. 20

D. 32

Ans: A

Explanation

160 >> 5 equals 160 / 2^5 = 5.

Q191 | Mixed | Bitwise Operators | Difficulty: Moderate

Fill in the blank: The expression x << n is functionally similar to multiplying x by _____.

A. 2 divided by n

B. 2 raised to the power n

C. n raised to the power 2

D. n itself

Ans: B

Explanation

Left shifting by n positions multiplies the value by 2^n.

Q192 | Mixed | Bitwise Operators | Difficulty: Moderate

Which of the following correctly checks if left-shifting 4 by 2 gives exactly 16?

A. (4 >> 2) == 16

B. 4 << 2 = 16

C. (4 << 2) == 16

D. (4 << 16) == 2

Ans: C

Explanation

4 << 2 evaluates to 16, so comparing it with == 16 using proper syntax correctly checks the condition.

Q193 | Mixed | Bitwise Operators | Difficulty: Moderate

What is the output of: printf("%d", 9 << 1);

A. 9

B. 4

C. 18

D. 20

Ans: C

Explanation

9 << 1 equals 9 x 2 = 18.

Q194 | Mixed | Bitwise Operators | Difficulty: Moderate

Which of the following is TRUE regarding bit shift operators?

A. They are binary operators that reposition the bits of their left operand

B. They always produce a value of zero

C. They cannot be combined with relational operators

D. They can only be used with negative numbers

Ans: A

Explanation

Shift operators take two operands and move bits of the left operand by the number of positions given by the right operand; they can be freely combined with relational operators.

Q195 | Mixed | Bitwise Operators | Difficulty: Moderate

Which statement about the limitations of shift operators is accurate?

A. Shift operators automatically prevent overflow

B. Shift operators can only be used once per program

C. Shifting by a count equal to or greater than the type's bit-width can produce unpredictable results

D. Shift operators never have any limitations

Ans: C

Explanation

Excessively large shift counts relative to the data type's size lead to undefined or unpredictable behavior.

Q196 | Mixed | Bitwise Operators | Difficulty: Moderate

What is the value of 256 >> 8?

A. 256

B. 2

C. 1

D. 0

Ans: C

Explanation

256 >> 8 equals 256 / 2^8 = 1.

Q197 | Mixed | Bitwise Operators | Difficulty: Moderate

Assertion (A): Shift operators are useful for fast multiplication and division by powers of two. Reason (R): Shifting bits changes their place value by a factor of two per position.

A. Both A and R are true, but R does not explain A

B. Both A and R are true, and R correctly explains A

C. A is true, but R is false

D. A is false, but R is true

Ans: B

Explanation

Because each shift changes place value by a factor of two, repeated shifts scale the value by powers of two — R correctly explains A.

Q198 | Mixed | Bitwise Operators | Difficulty: Moderate

What is the value of 3 << 6?

A. 48

B. 384

C. 192

D. 96

Ans: C

Explanation

3 << 6 equals 3 x 2^6 = 192.

Q199 | Mixed | Bitwise Operators | Difficulty: Moderate

What is the value of 192 >> 6?

A. 12

B. 3

C. 6

D. 1

Ans: B

Explanation

192 >> 6 equals 192 / 2^6 = 3.

Q200 | Mixed | Bitwise Operators | Difficulty: Moderate

Match the following:

```
(i) << operator   (ii) >> operator   (iii) == operator
(a) Relational comparison  (b) Left shift  (c) Right shift
```

A. (i)-a, (ii)-b, (iii)-c

B. (i)-c, (ii)-b, (iii)-a

C. (i)-b, (ii)-a, (iii)-c

D. (i)-b, (ii)-c, (iii)-a

Ans: D

Explanation

<< is the left shift operator (b), >> is the right shift operator (c), and == is a relational comparison operator (a).

Q201 | Mixed | Bitwise Operators | Difficulty: Moderate

What is the value of 45 >> 1?

A. 23

B. 90

C. 44

D. 22

Ans: D

Explanation

45 is 101101 in binary; shifted right by 1 gives 10110, i.e., 22 (the last bit, 1, is discarded).

Q202 | Scenario | Bitwise Operators | Difficulty: Moderate

An OAVS TGT exam question asks a candidate to reposition the bits of a value toward higher place values. Which operator should the candidate use?

A. >> (right shift)

B. != (relational)

C. == (relational)

D. << (left shift)

Ans: D

Explanation

Moving bits toward higher place values (i.e., increasing significance) is exactly what the left shift operator does.

Q203 | Scenario | Bitwise Operators | Difficulty: Moderate

An OAVS TGT exam question asks a candidate to reposition the bits of a value toward lower place values. Which operator should the candidate use?

A. != (relational)

B. << (left shift)

C. >> (right shift)

D. == (relational)

Ans: C

Explanation

Moving bits toward lower place values is exactly what the right shift operator does.

Q204 | Mixed | Bitwise Operators | Difficulty: Moderate

What will be printed by: printf("%d", (6 << 1) == 12 ? 1 : 0);

A. 1

B. 0

C. 12

D. 6

Ans: A

Explanation

6 << 1 equals 12, so the condition (6 << 1) == 12 is true, and the conditional expression evaluates to 1.